

A blue-tinted background image showing a group of business professionals in a meeting. One person is writing on a document, another is holding a tablet, and others are looking at documents or devices. The scene is professional and collaborative.

Agile Methodology

Contents

1. Introduction
2. Agile Overview
3. 12 Agile Principles
4. Agile Development Cycle
5. Advantages & Disadvantages of Agile
6. Top Methodologies Used to Implement Agile
7. Top Methodologies Used to Implement Agile - Scrum
8. Top Methodologies Used to Implement Agile - Kanban
9. How to Get Started With Agile
10. Finding the Best Agile Tool

Everyone manages projects

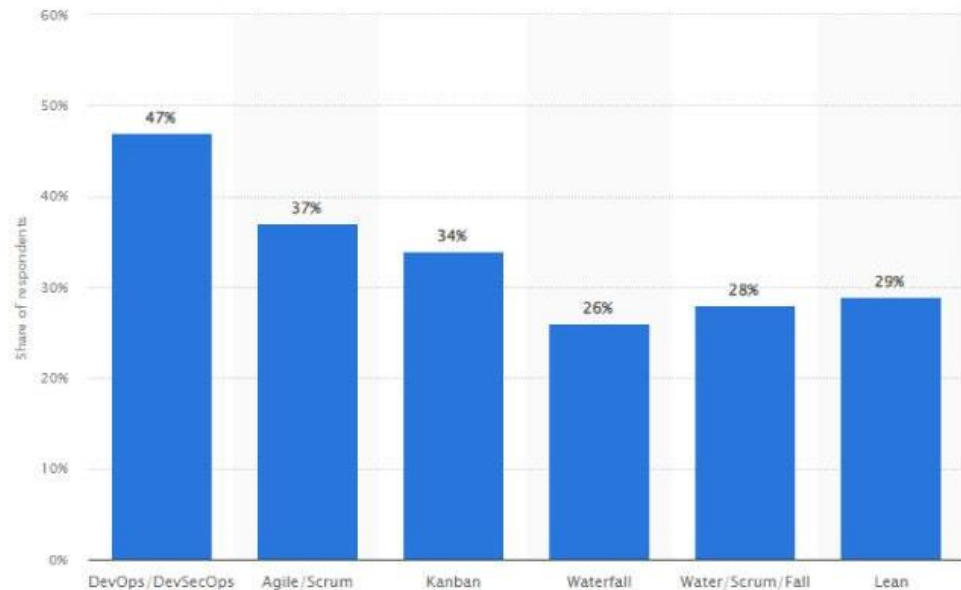
Whether they're a certified project manager or not.

- Many people manage projects without formal training, often “going with the flow”.
- This can lead to challenges in managing multiple projects, meeting deadlines, and adapting to changes.
- Companies use a standard project management methodology, but projects still fail.
- Adopting a standardized approach like Agile is recommended for anyone managing work.

Facts

Breakdown of software development methodologies practiced in 2022.

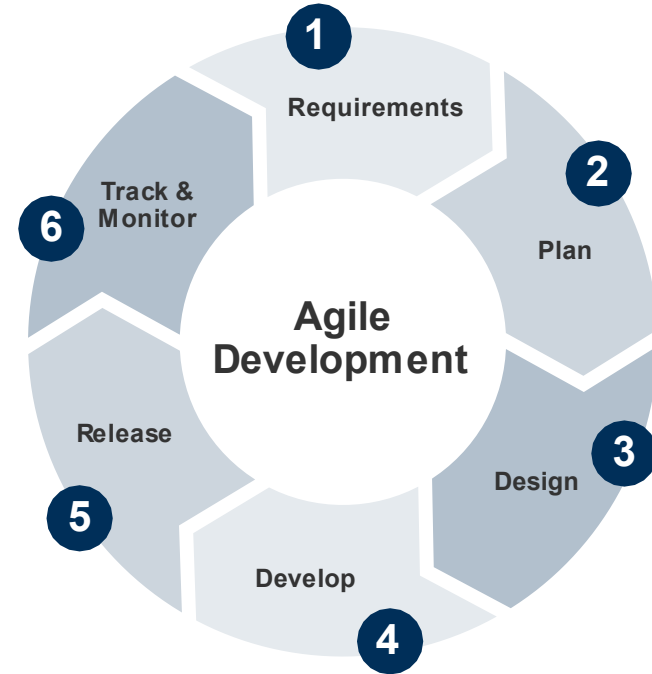
- Do you want to choose the best method for your software development?
- According to a report by Statista, in 2022, 37% of respondents use Agile methodology, while 26% use Waterfall methodology.



Agile Overview

Agile project management is based on an incremental, iterative approach

Instead of in-depth planning at the beginning of the project, Agile methodologies are open to changing requirements over time and encourages constant feedback from the end users. The goal of each iteration is to produce a working product.



Agile refers to any process that aligns with the concepts of the Agile Manifesto. In 2001, 17 software developers met to discuss lightweight development methods. They published the **Manifesto for Agile Software Development**, which covered how they found “better ways of developing software by doing it and helping others do it.”

12 Agile Principles

The Agile Manifesto lists 12 principles to guide teams on how to execute with agility:

1 Satisfy the customer

Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

2 Welcome changing requirements

Even late in development. Agile processes harness change for the customer's competitive advantage.

3 Deliver working software frequently

From a couple of weeks to a couple of months, with preference to the shorter timescale.

4 Work together

Business people and developers must work together daily throughout the project.

5 Build projects around motivated individuals

Give them the environment and support they need, and trust them to get the job done.

6 Face-to-face conversation

The most efficient way to communicate within a development team is face-to-face conversation.

12 Agile Principles

7 Focus on working software

Working software is the primary measure of progress.

8 Promote sustainable development

Allowing sponsors, developers, and users to maintain a constant pace indefinitely.

9 Ensure technical excellence

Continuous attention to technical excellence and good design enhances agility

10 Simplicity

The art of maximizing the amount of work not done – is essential.

11 Self-organizing teams

The best architectures, requirements, and designs emerge from self-organizing teams.

12 Reflect and adjustment

The team regularly reflects on improving and adjusts its behavior accordingly.

Lets discuss

What are the key characteristic of a self-organizing team?

- Self-organizing teams take ownership of their tasks, deciding how best to accomplish their goals without external micromanagement.
- Key Characteristics:
 - **Autonomy:** Teams decide how to approach their work within the boundaries of the project's goals and constraints.
 - **Accountability:** Members are collectively responsible for delivering results.
 - **Collaboration:** Team members collaborate to distribute tasks based on skills and interests, ensuring optimal performance.
 - **Continuous Learning:** They adapt processes and practices based on retrospective insights, improving over time.
- **Analogy:** Imagine a jazz band. Each musician knows their instrument (specialty) and works together without a conductor. They listen to one another, adapt their playing, and adjust to create harmonious music (product delivery).

Agile Development Cycle

The phases in the Agile development cycle may not happen in succession; they are flexible and always evolving, with many happening in parallel.

Planning

Once an idea is deemed viable, the project team comes together to identify features, prioritize each feature, and assign them to an iteration.

Requirements analysis

Key stakeholders and users meet to identify business requirements that are quantifiable, relevant, and detailed.

Design

The design is prepared from the requirements identified and the team considers what the product or solution will look like, deciding on a test strategy or plan to proceed.

Implementation, coding, development

Coding or developing features, scheduling iterations for deployment.

Testing

Test the code against the requirements to make sure the product is actually solving customer needs. This phase includes unit testing, integration testing, system testing, and acceptance testing.

Deployment

Deliver the product to customers. Once customers start using the product, they may run into new problems that the project team will need to address in future iterations.

Advantages of Agile

Agile evolved from different development approaches in the 1990s and is a response to some project managers' dislike of the rigid, linear Waterfall methodology. It focuses on flexibility, continuous improvement, and speed.

+ **Change is embraced**

With shorter planning cycles, there's always opportunity to refine and reprioritize the backlog to accommodate changes throughout the project.

+ **End-goal can be unknown**

Agile is beneficial for projects where the end-goal is not clearly defined. As the project progresses, the goals will become evident and the team can adapt.

+ **Faster, high-quality delivery**

Breaking down the project into iterations allows the team to focus on high-quality development, testing, and collaboration. Conducting testing during each iteration means that bugs get identified and solved more quickly.

+ **Strong team interaction**

Agile embraces frequent communication and face-to-face interactions.

+ **Customers are heard**

Customers have many opportunities to see the work being delivered, share their input, and have an impact on the end product.

+ **Continuous improvement**

Feedback is encouraged from users and team members throughout the project, so lessons learned are used to improve future iterations.

Disadvantages of Agile

While flexibility in Agile is usually a positive, it also comes with some trade-offs. It can be hard to establish a solid delivery date, documentation can be neglected, or the final product can be very different than originally intended.

- **Planning can be less concrete**

Because project managers are often reprioritizing tasks, it's possible some items scheduled for delivery may not be complete in time. And, additional sprints may be added at any time in the project, adding to the overall timeline.

- **Team must be knowledgeable**

Agile teams are usually small, so team members must be highly skilled in a variety of areas and understand Agile methodology.

- **Time commitment from developers is required**

Active involvement and collaboration is required throughout the Agile process, which is more time consuming than a traditional approach.

- **Documentation can be neglected**

Agile prefers working deliverables over comprehensive documentation. While documentation on its own does not lead to success, teams should find the right balance between documentation and discussion.

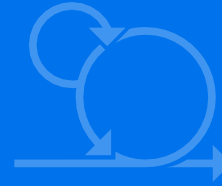
Lets discuss

How can Agile teams balance the principle of 'working software over comprehensive documentation' with the long-term need for maintainable and well-documented systems?

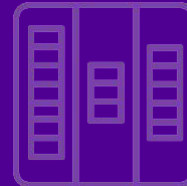
- **Balance Through Contextual Documentation:** Agile doesn't reject documentation entirely; instead, it promotes documentation that is "just enough" to meet the team's and stakeholders' needs. For example:
 - Critical areas like APIs, architecture, or complex business rules may need documentation to ensure system maintainability.
 - A user manual might be necessary for end-users.
- **Focus on Value:** Teams should continuously ask, "Who needs this documentation, and how will they use it?" Documentation should support value delivery rather than delay it.
- **Tools and Collaboration:** Modern tools like wikis, code comments, and shared repositories allow lightweight, iterative, and easily updated documentation.
- **Risk Awareness:** Skipping too much documentation can lead to risks, especially in long-lived projects or when team members change. Teams should weigh the risks of insufficient documentation against the benefits of delivering software quickly.

Top Methodologies Used to Implement Agile

There are a number of specific methods within the Agile movement. We'll take an in-depth look at two of the top Agile Methodologies: Scrum and Kanban.



Scrum



Kanban

Scrum Methodology

Scrum is a subset of Agile and one of the most popular process frameworks for implementing Agile.

It is an iterative development model often used to manage complex software and product development. Fixed-length iterations, called sprints lasting 2 to 4 weeks long, allow the team to ship software on a regular cadence. At the end of each sprint, stakeholders and team members meet to plan next steps.



Roles in Scrum



Product Owner

The Scrum Product Owner has the vision of what to build and conveys that to the team. He or she focuses on business and market requirements, prioritizing the work that needs to be done, managing the backlog, providing guidance on which features to ship next, and interacting with the team and other stakeholders to make sure everyone understands the items on the product backlog.

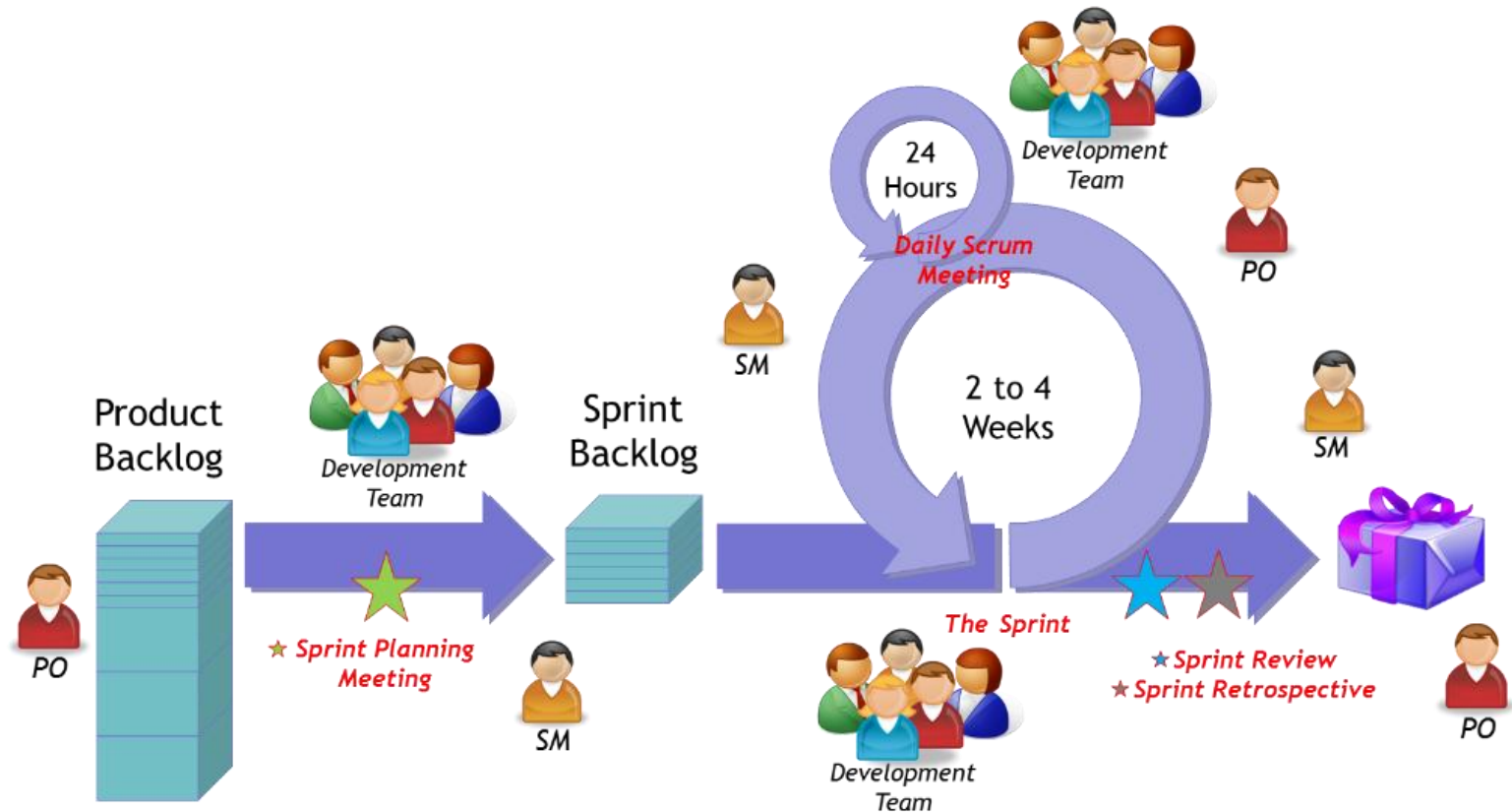
Scrum Master

Often considered the coach for the team, the Scrum Master helps the team do their best possible work. This means organizing meetings, dealing with roadblocks and challenges, and working with the Product Owner to ensure the product backlog is ready for the next sprint.

Scrum Team

The Scrum Team is comprised of five to seven members. Unlike traditional development teams, there are not distinct roles like programmer, designer, or tester. Everyone on the project completes the set of work together.

Steps in the Scrum Process



Steps in the Scrum Process

1 Product backlog

The product backlog is not a list of things to be completed, but rather it is a list of all the desired features for the product.

2 Sprint planning

Before each sprint, the Product Owner presents the top items on the backlog in a sprint planning meeting. The team determines the work they can complete during the sprint and moves the work from the product backlog to the sprint backlog.

3 Sprint Backlog

A sprint backlog is a list of work items your team plans to complete during a project sprint. A clear sprint backlog prevents scope creep by clarifying exactly what your team will be doing—and not doing—during each sprint.

4 Daily Scrum meetings

The Daily Scrum is a 15-minute stand-up meeting that happens at the same time and place every day during the sprint. During the meeting each team member talks about what they worked on the day before, what they'll work on today, and any roadblocks.

5 Sprint review meeting

At the end of each sprint, the team presents the work they have completed as a live demo rather than a presentation.

6 Sprint retrospective meeting

Also at the end of each sprint, the team reflects on how well Scrum is working for them and talks about any changes that need to be made in the next sprint.

Lets discuss

Is PO a real user or a member of the development team?

- The **PO** is a representative of the business side or the stakeholders.
- They own the product backlog and are responsible for defining and prioritizing features based on the business goals and user needs.
- They ensure the team builds the right product and delivers maximum value.
- Relationship with Real Users
 - While the PO is not usually a real user, they should have a deep understanding of the users' needs.
 - They may gather insights by:
 - Interacting directly with users through interviews, surveys, or feedback sessions.
 - Collaborating with stakeholders who have close contact with end-users (e.g., marketing or customer support teams).
- Relationship with the Development Team
 - The PO works closely with the team but is not a developer.
 - They clarify the "what" and "why" of features, while the development team decides "how" to implement them

Tools and Artifacts in Scrum

In addition to roles and ceremonies, Scrum projects also include certain tools and “artifacts”. For example, the team uses a Scrum board to visualize the backlog or a burndown chart to show outstanding work. The most common are:



Scrum board

The Scrum board helps to visualize your sprint backlog and traditionally involves index cards or Post-it notes on a whiteboard. The board is usually divided into three categories: to do, work in progress, and done. The team updates the board by moving tasks (written on cards) through the columns on the board.



User stories

A user story describes a software feature from the customer's perspective. It includes the type of user, what they want, and why they want it.



Burndown chart

A burndown chart represents all outstanding work. The backlog is usually on the vertical axis, with time along the horizontal axis. A burndown chart can warn the team of potential risk and helps to show the impact of decisions.



Timeboxing

A timebox is a set period of time that a team works towards completing a goal. Instead of letting a team work until the goal is reached, the timebox approach stops work when the time limit is reached.



Icebox

Any user stories that are recorded but not moved to development are stored in the icebox.

Example of a Scrum Board

Scrum Board

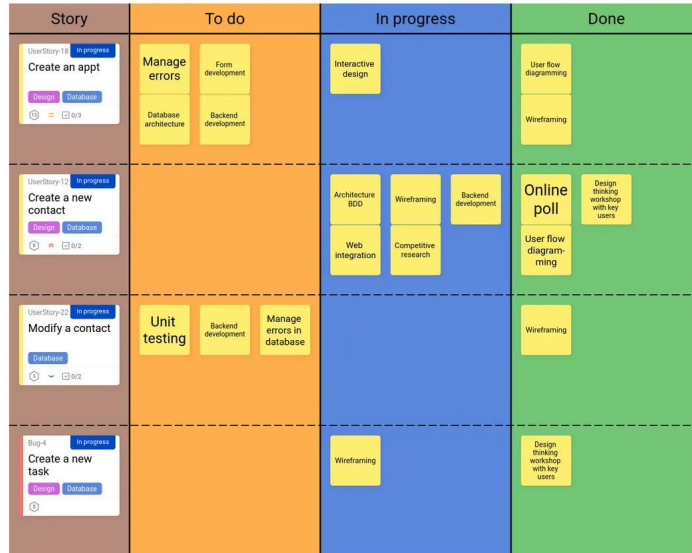
Team Agile Digest - Sprint 25

From 09/12 to 16/12

Sprint day(s) remaining: 3

Backlog

- UserStory-1 **Ready**
Classify emails
Database Design
- UserStory-8 **Ready**
Add an attachment to an email
Design Database
- UserStory-12 **Ready**
Delete a contact
Database
- UserStory-15 **To specify**
Propose several slots to schedule an appt
- UserStory-18 **To specify**
Share a task
- UserStory-21 **To specify**
Modify an appt
- UserStory-22 **To specify**
Share an appt
- UserStory-23 **To specify**
Assign a task to a contact



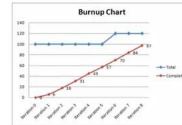
Key



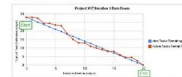
Sprint Goals:

- Doubling the number of beta testers
- Increasing the number of new contacts created by 35%

Release Burn Up Chart



Burn Down Chart



Core Hour Discussion Area

- UserStory-12: Delay to complete wireframing
- UserStory-25: Doubts with the user sample for the design thinking workshop
- UserStory-18: Difficulties to deal with duplications in database

Tracking of absences

	Mo	Tu	We	Th	Fr	Mo	Tu	We	Th	Fr
Sam		RW			PH					
Alex				RW	PH					
Sandra			RW		PH					
Steph					PH	Vac	Vac	Vac	Vac	Vac
Clara					PH					

Legend

- RW: Remote working
- Vac: Vacation
- PH: Public holidays

Tracking of impediments

In progress	Resolved
Impossible to access database in production configuration	Customer data recovery non available
Waiting feedback from marketing dept on ST-5	

Reality



User Story

A user story is a simple description of a feature or functionality written from the perspective of a customer or end-user.

The common user story template would look something like this:

As a **[type of user]**,
I want **[an action]**
so that **[a benefit/value]**.

Acceptance criteria outline what conditions are met for it to be considered done

Example:

- **User Story:** Login into the System
 - As a user, I want to log into the system using my credentials so that I can access my account securely.
- **Acceptance Criteria:**
 - Login form includes fields for username/email and password.
 - Valid credentials are required for successful login.
 - Display error message ("Invalid username or password") for incorrect credentials.
 - Passwords are masked during typing. Provide a "Forgot Password" link for account recovery.
 - Redirect users to the dashboard after successful login.
 - Redirect to login page if the session expires.
 - Lock account after 5 failed attempts, showing an "Account locked" message.

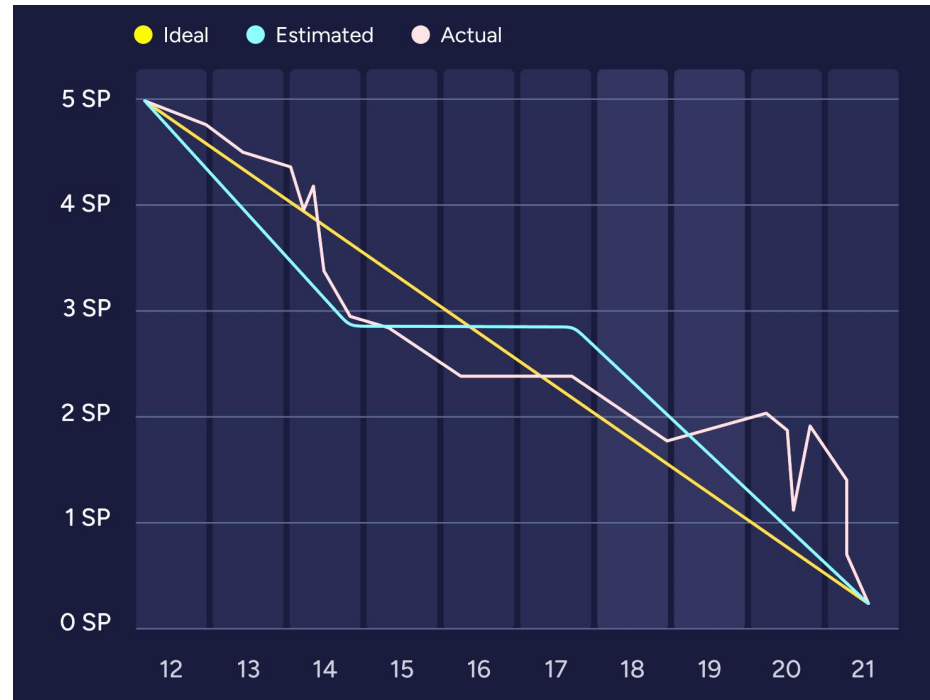
Burndown chart

An Agile burndown chart is a simple graph that helps you measure a project's progress against time. It visually represents the amount of work remaining over time in a sprint.

Agile burndown charts are valuable tools for visualizing work completion, predicting project timelines, identifying bottlenecks, and making informed decisions to ensure successful project delivery.

- Key components of an Agile burndown chart:
 - **X-axis (Time):** Time left until the project deadline, in days or sprints.
 - **Y-axis (Work Remaining):** Tasks or story points left to complete.
 - **Start Point:** Total estimated work at the project's start.
 - **Finish Point:** Predicted completion based on estimates and efficiency.
 - **Ideal Line:** Straight line showing expected progress for on-time delivery.
 - **Actual Line:** Real-time progress tracking.
 - **Ahead of Schedule:** Actual line below the ideal line.
 - **Behind Schedule:** Actual line above the ideal line, signaling potential delays.

Example of Burndown Chart



Benefits and Limitations of Burndown Chart

Benefits of an Agile burndown chart:

- **Progress Tracking:** Monitors remaining work and predicts completion.
- **Early Warnings:** Flags deviations for timely corrective actions.
- **Informed Decisions:** Highlights the impact of decisions for better planning.
- **Clear Communication:** Keeps team and stakeholders aligned on progress.
- **Accurate Estimation:** Predicts completion based on trends.
- **Team Motivation:** Boosts performance through visible progress.
- **Transparency:** Compares planned vs. actual progress clearly.
- **Enhanced Collaboration:** Aligns everyone on project status and goals.

Limitations of an Agile burndown chart:

- **Quality Blindness:** Focuses on quantity, not work quality.
- **Challenges Ignored:** Doesn't reflect project-specific obstacles.
- **Complexity Limits:** Less effective for highly complex projects.
- **Pressure on Teams:** May demotivate if progress lags.
- **Steep Learning Curve:** Teams new to Agile may struggle with accurate estimation.
- **Scope Changes:** Struggles to adapt to significant project changes.

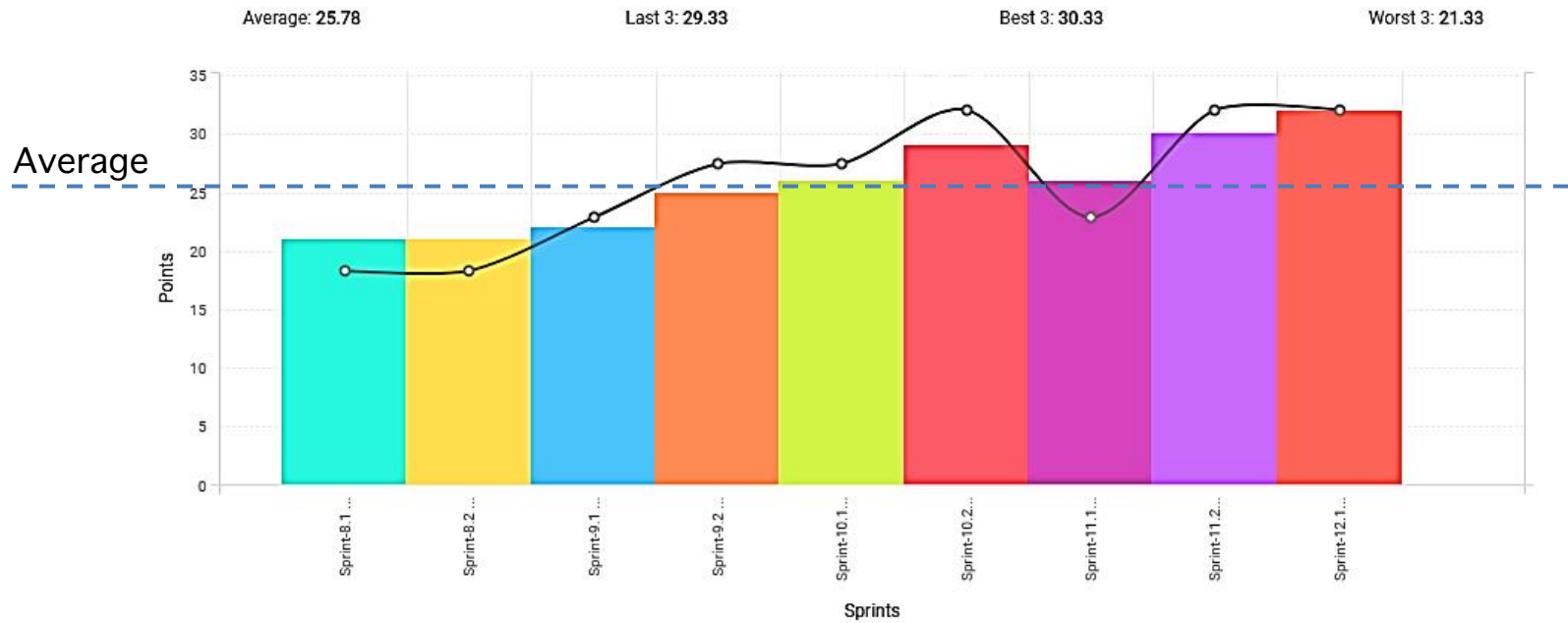
Agile Velocity

An Agile burndown chart is a simple graph that helps you measure a project's progress against time. It visually represents the amount of work remaining over time in a sprint.

A story point is a unit of measurement that estimates how much effort is required to complete a user story based on complexity, risk, and repetition.

- **Assign Story Points:** Before the sprint starts, assign story points to each task based on difficulty. Experienced teams use past velocity data to estimate the total story points the team can complete in a sprint. Example:
 - Task 1: 5 points
 - Task 2: 3 points
 - Task 3: 9 points
 - Task 4: 6 points
 - Task 5: 4 points
- **Calculate Sprint Velocity:** At the end of the sprint, add up the story points of completed tasks only. Incomplete tasks are not included.
 - Example: If Tasks 1, 3, 4, and 5 are completed, $\text{velocity} = 5 + 9 + 6 + 4 = 24$.
- **Determine Average Velocity:** Repeat over multiple sprints. Add the total velocity from all sprints and divide by the number of sprints.
 - Example: Velocities = 24, 18, 26, 20. $\text{Average velocity} = (24 + 18 + 26 + 20) / 4 = 22$.

Example of Agile Velocity



Insights from Burndown Chart and its Limitations

Insights from Agile Velocity

- **Understand Workload Capacity:** A steady increase in velocity shows improved efficiency, while a decrease may indicate burnout or excessive workload.
- **Spot Bottlenecks:** Significant drops in velocity or fluctuations in velocity charts often reveal organizational issues or sprint-specific problems.
- **Identify Improvement Areas:** Downward trends in velocity charts highlight where numbers are dropping, enabling timely investigation and adjustments.

Limitations of Agile Velocity

- **Doesn't Measure Work Quality:** High velocity may mask poor-quality work, such as bugs or dissatisfied customers.
- **Encourages Speed Over Quality:** Overemphasis on velocity can lead to rushed work, reduced quality, and burnout.
- **Not Comparable Across Teams:** Velocity is team-specific due to varying methods of assigning story points, making comparisons meaningless.

Lets discuss

Should Agile teams use burndown charts and velocity as the primary measures of success, or do these metrics risk undermining true Agile values?

Yes, use it

- **Progress Transparency:** Burndown charts visually track progress, while velocity helps estimate future capacity.
- **Accountability:** Metrics help teams focus on delivering consistent work and identifying inefficiencies.
- **Predictability:** They provide stakeholders with a clear view of progress and delivery timelines.

No, don't use it

- **Risk of Gaming the System:** Teams may prioritize metrics over quality, sacrificing value.
- **Neglect of Agile Principles:** Focusing on velocity can shift attention from customer value to just "burning down points."

Balanced Approach

- **Misleading Metrics:** Velocity doesn't guarantee the right work, and burndown charts can be misleading if priorities change.
- **Complement Metrics with Value-Driven Discussions:** Use metrics as tools for reflection, not as goals.
- **Focus on Outcomes, Not Output:** Pair metrics with qualitative feedback to ensure alignment with customer needs.
- **Avoid Metric Worship:** Use metrics to guide conversations,

Advantages of Scrum

Scrum is a highly prescriptive framework with specific roles and ceremonies. While it can be a lot to learn, these rules have multiple advantages:

- + **More transparency and project visibility**

With daily stand-up meetings, the whole team knows who is doing what and issues are identified in advance, improving communication and enabling the team to take care of issues right away.

- + **Increased team accountability**

There is no project manager. Instead, the team collectively decides what work they can complete in each sprint, working together collaboratively, with accountability.

- + **Easy to accommodate changes**

With short sprints and constant feedback, it's easier to accommodate changes.

- + **Increased cost savings**

Constant communication ensures the team is aware of all issues and changes sooner, helping to lower expenses and increase quality.

Disadvantages of Scrum

While Scrum offers some concrete benefits, it also has some downsides. Scrum requires a high level of experience and commitment from the team and projects can be at risk of scope creep:

- **Risk of scope creep**

Some Scrum projects can experience scope creep due to a lack of a specific end date, tempting stakeholders to keep requesting additional functionality.

- **Team requires experience and commitment**

The team needs to be familiar with Scrum principles to succeed, as well as needs to commit to the daily meetings and stay on the team for the entire project.

- **The wrong Scrum Master can ruin everything**

The Scrum Master is very different from a project manager. The Scrum Master does not have authority over the team, so he or she must trust the team to complete the work.

- **Poorly defined tasks can lead to inaccuracies**

Project costs and timelines won't be accurate if tasks are not well defined. If the initial goals are unclear, planning becomes difficult and sprints can take more time than originally estimated.

Lets discuss

Should the Scrum Master have the authority to remove a team member if they are consistently disrupting the Scrum process?

- The Scrum Master is a servant-leader and facilitator, but their authority over the team is deliberately limited.
- **Scrum Framework Perspective:**
 - The Scrum Master is not a traditional manager; they are responsible for facilitating the process, removing impediments, and coaching the team. Removing a team member is not within their direct authority.
- **Collaborative Approach:**
 - A disruptive team member should be addressed through coaching, conflict resolution, and encouraging self-organization within the team.
 - The Scrum Master should work with stakeholders or line managers (who have the authority) to discuss potential removal if coaching fails.
- **When Removal is Necessary:**
 - In extreme cases where disruption jeopardizes the team's ability to deliver, the Scrum Master must escalate the issue to appropriate management, emphasizing the impact on team performance and sprint goals.

Kanban Methodology

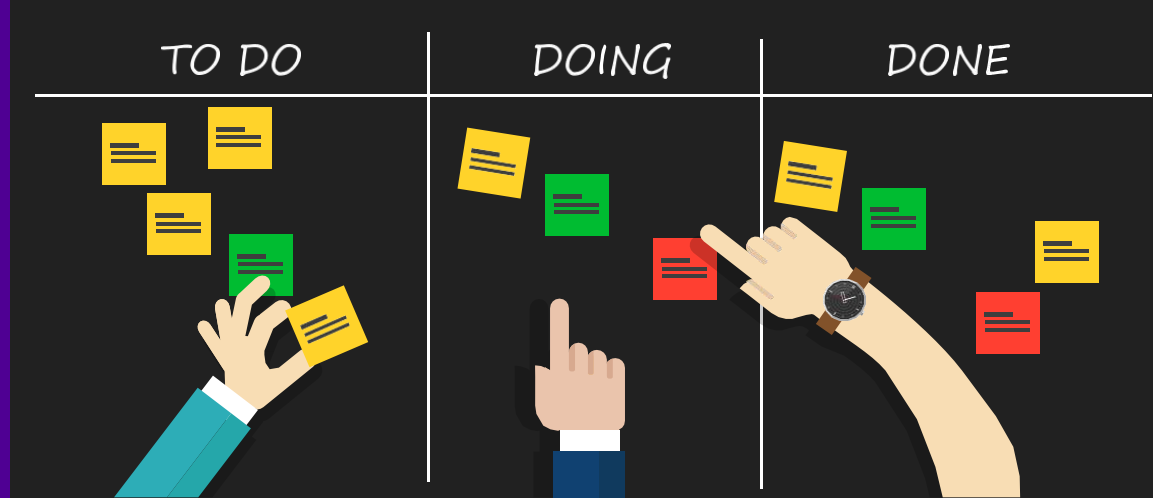
Kanban is Japanese for “visual sign” or “card.” It is a visual framework used to implement Agile and shows what to produce, when to produce it, and how much to produce. It encourages small, incremental changes to your current system and does not require a certain set up or procedure (meaning, you could overlay Kanban on top of other existing workflows).

Kanban was inspired by the Toyota Production System and Lean Manufacturing. In the 1940s, Toyota improved its engineering process by modeling it after how supermarkets stock shelves. Engineer Taiichi Ohno noticed that supermarkets stock just enough product to meet demand. Inventory would only be restocked when there was empty space on the shelf (a visual cue). These same ideas apply to software teams and IT projects today. In this context, development work-in-progress (WIP) takes the place of inventory, and new work can only be added when there is an “empty space” on the team’s visual Kanban board. Kanban matches the amount of WIP to the team’s capacity, improving flexibility, transparency, and output.

About the Kanban Board

A Kanban board is a tool to implement the Kanban method for projects.

Traditionally, this tool has been a physical board, with magnets, plastic chips, or sticky notes on a whiteboard. However, in recent years, more and more project management software tools have created online Kanban boards.



A Kanban board, whether it is physical or online, is made up of different swim lanes or columns. The simplest boards have three columns: to do, in progress, and done. Other projects may consist of backlog, ready, coding, testing, approval, and done columns.

Kanban cards (like sticky notes) represent the work and each card is placed on the board in the lane that represents the status of that work. These cards communicate status at a glance. You could also use different color cards to represent different details. For example, green cards could represent a feature and orange cards could represent a task.

Core Practices and Principles of Kanban

Every Kanban project should follow these core principles:



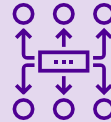
Visualize the workflow

A visual representation of your work allows you to understand the big picture and see how the flow of work progresses. By making all the work visible you can identify issues early on and improve collaboration.



Limit work in progress (WIP)

Work in progress limits determine the minimum and maximum amount of work for each column on the board or for each workflow. By putting a limit on WIP, you can increase speed and flexibility, and reduce the need for prioritizing tasks.



Manage and enhance the flow

The flow of work throughout the Kanban board should be monitored for possible improvements. A fast, smooth flow shows the team is creating value quickly.



Make process policies explicit

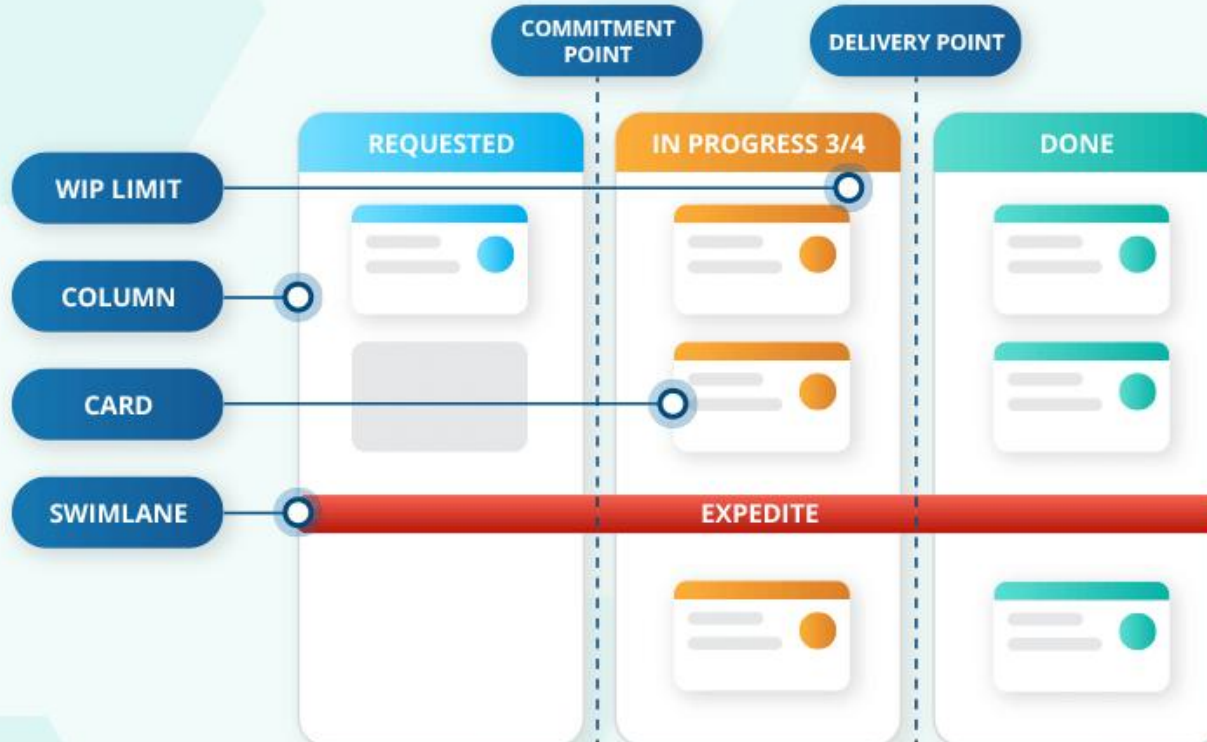
Everyone needs to understand how things work or what qualifies as “done”. Modify the board to make these processes more clear.



Continuously improve

The Kanban method encourages small, continuous changes that stick. Once the Kanban system is in place, the team will be able to identify and understand issues and suggest improvements.

KANBAN BOARD



Advantages of Kanban

Kanban's visual nature offers a unique advantage when implementing Agile. The Kanban board is easy to learn and understand, it improves flow of work, and minimizes cycle time:

+ Increases flexibility

Kanban is an evolving, fluid model. There are no set phase durations and priorities are reevaluated with new information.

+ Reduces waste

Kanban revolves around reducing waste, ensuring that teams don't spend time doing work that isn't needed or doing the wrong kind of work.

+ Easy to understand

The visual nature of Kanban helps to make it intuitive and easy to learn.

+ Improves delivery flow

Kanban focuses on the just-in-time delivery of value and delivering work to customers on a regular cadence.

+ Minimizes cycle time

Cycle time is the amount of time it takes for work to move through the team's workflow. In Kanban projects, the entire team helps to ensure the work is moving quickly and successfully through the process.

Disadvantages of Kanban

Many of the disadvantages associated with Kanban come with misuse or mishandling of the Kanban board. An outdated or overcomplicated board can lead to confusion, inaccuracies, or miscommunication:

- **Outdated board can lead to issues**

The team must be committed to keeping the Kanban board up to date, otherwise they'll be working off inaccurate information.

- **Teams can overcomplicate the board**

The Kanban board should remain clear and easy to read. Adding bells and whistles to the Kanban board just buries the important information.

- **Lack of timing**

The columns on the Kanban board are marked by phase, with no timeframes associated.



Lets discuss

Does imposing strict Work-In-Progress (WIP) limits in Kanban stifle creativity and productivity, or does it enhance team efficiency?

- Arguments for WIP Limits:
 - Enhanced Focus: Teams concentrate on finishing work rather than juggling multiple items, reducing context-switching and errors.
 - Improved Flow: Help identify bottlenecks in the workflow, enabling the team to address inefficiencies and deliver faster.
 - Encourages Collaboration: When hit WIP limits, they are more likely to assist others, fostering teamwork.
- Arguments Against WIP Limits:
 - Perceived Restriction: Some team members may feel that WIP limits constrain their ability to multitask or explore creative solutions.
 - Dynamic Work Environments: In fast-paced or innovative teams, strict WIP limits might hinder adaptability, especially when priorities shift rapidly.
 - Psychological Resistance: Teams unfamiliar with Kanban may resist WIP limits, perceiving them as artificial constraints imposed on their workflow.
- Middle ground: Flexible and regular review WIP Limits

Other Agile Methodologies

Extreme Programming (XP)

This type of software development is intended to improve quality and responsiveness to evolving customer requirements.

Feature-driven development (FDD)

There are five basic activities in FDD: develop overall model, build feature list, plan by feature, design by feature, and build by feature.

Adaptive system development (ASD)

ASD represents the idea that projects should always be in a state of continuous adaptation, and has a cycle of three repeating series: speculate, collaborate, and learn.

Dynamic Systems Development Method (DSDM)

DSDM addresses the common failures of IT projects. The eight principles of DSDM are: focus on the business need, deliver on time, collaborate, never compromise quality, build incrementally from firm foundations, develop iteratively, communicate continuously and clearly, and demonstrate control.

Lean Software Development (LSD)

LSD can be characterized by seven principles: eliminate waste, amplify learning, decide as late as possible, deliver as fast as possible, empower the team, build integrity in, and see the whole.

Crystal Clear

This methodology can be used with teams of six to eight developers and it focuses on the people, not processes or artifacts. Crystal Clear requires the following: frequent delivery of usable code to users, reflective improvement, and osmotic communication preferably by being co-located.

How to Get Started with Agile

Getting started with Agile can be easy. We'll take a look at a few ways to implement Agile practices and how to pick the right Agile tool.



3 Ways to Implement Agile Practices

Whichever route you choose, remember that Agile is flexible in its very nature. There is no wrong or right way to get started with Agile.

Stand up Meetings

A simple way to get started with Agile is to incorporate daily stand-up meetings into your project. Daily stand-up meetings are easy to incorporate into any project methodology you already use (even Waterfall) and don't require any training or knowledge transfer.

Kanban Board

Another way to incorporate Agile practices is to create and use a Kanban board. The Kanban board is a simple tool to help your team visualize the flow of work as it's getting done. Use the board during stand-up meetings to discuss current work in progress or display it where your team can easily access it to make updates to task status.

Changing Team Roles

Some methods of Agile may result in the need to change team roles. For example, working with Scrum, the team may need to take more responsibility and boost speed of delivery. A good place to start with Scrum is to talk about the roles and responsibilities. Every project must have a Scrum Master, Product Owner, and Scrum Team. Clarifying these roles will help teams understand their responsibilities and remain accountable.

Finding the Best Agile Tool

If you're considering a switch to Agile project management, you should determine which tool is best to help track and manage your projects. Here are the top things to consider when choosing a tool:



Familiar and Easy to Use

When switching to a new tool, familiarity is key. You don't want your team spending valuable time learning a new program. Agile project management software should be flexible and intuitive to use.



Collaboration & Communication

Find a tool that facilitates collaboration between internal and external stakeholders. Cloud tools enable people to work in real-time, view and edit projects from anywhere, stay up to date on current status, and communicate with others viewing the project.



Searchable, Central Storage

It's increasingly important to use cloud storage services like Dropbox, Google Drive, Box, or OneDrive for project documentation storage. Whatever service you use, make sure your Agile tool can seamlessly connect with them to store vital assets.



Mobile Ready

The modern worker relies on their smartphone, tablet and other mobile devices to get work done. And many workers want to access parts of their project management software while they are on the go. Be sure to evaluate all the mobile options a tool has to offer.



Work Visualization

Today, work design is infinitely different. The "one-size-fits-all" approach to project management no longer works. Find a tool that includes multiple ways to visualize and manage your work - whether it be waterfall or agile - with Gantt charts, spreadsheet, kanban boards, calendars, reports, and dashboards.

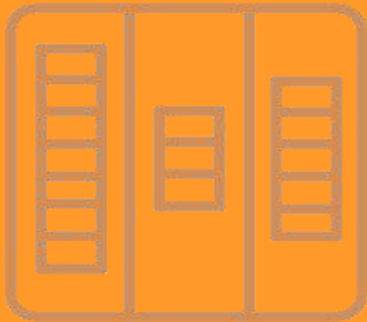
A More Agile Way to Manage Work



With fewer than one-third of all projects completed on time and on budget, it can sometimes seem impossible to lead new projects to success.

That's why finding the best project management method is important to the success of your projects. Whether you decide to go with Agile, Scrum, Kanban, or even a hybrid of multiple methods, deciding on a standard way to manage projects will help to streamline the process, increase team output, and keep projects on track. Plus, once you've picked the best method for your team, finding the right agile project management tool can help implement and track the process and ensure no detail is missed.

Trello as Agile Project Management Tool



Next week, we will be conducting a hands-on lab using Trello as a project management tool. In this session, we will specifically implement a Scrum board to develop a simple software application. We will simulate various software development scenarios to explore how Trello can assist in managing the project.

Please make sure to register for a Trello account at www.trello.com beforehand. During the lab, we will dive directly into creating a board and its components. If you encounter any issues with registering, feel free to reach out in the WhatsApp group for assistance.

A blue-tinted background image showing a group of business professionals in a meeting. One person is writing on a document, another is holding a tablet, and a calculator is visible on the table.

Thank you

References

- Patel, R. (2022, October 26). Agile vs Waterfall - A Detailed Comparison Guide. Space-O Technologies (Canada). <https://www.spaceo.ca/blog/agile-vs-waterfall/>
- Patel, R. (2023, June 23). What is the Agile Manifesto? [Key Agile Values and Principles]. Space-O Technologies (Canada). <https://www.spaceo.ca/blog/agile-manifesto/>
- The history of Agile. (2019, November 21). Planview. <https://www.planview.com/resources/guide/agile-methodologies-a-beginners-guide/history-of-agile/>
- Richter, L. (n.d.). Real-Life agile project management & famous success stories. Smartsheet. <https://www.smartsheet.com/content/agile-project-management-examples#real-life-agile-project-examples-from-large-companies>